

10_DFS 与 BFS 图搜索

10 DFS 与 BFS 图搜索

题目原文

简述深度优先和广度优先算法的思想，采用该算法思想能解决经典的图上搜索问题：topo 序、任务调度、关键路径和强联通区域问题

考查类型

概念题和算法设计题。

DFS 思想

深度优先搜索从一个起点出发，沿着一条路径尽量向深处走；当没有未访问邻接点时，再回退到上一个结点继续搜索。

实现方式：

递归
显式栈

典型记录：

visited[v]
discover time
finish time
parent[v]

时间复杂度：

$O(V + E)$

BFS 思想

广度优先搜索从起点出发，先访问距离为 1 的点，再访问距离为 2 的点，按层推进。

实现方式：

队列

典型记录:

```
visited[v]
distance[v]
parent[v]
```

时间复杂度:

$O(V + E)$

BFS 可求无权图中从起点到各点的最短路径长度。

topo 序

拓扑序用于有向无环图。

DFS 做法:

1. 对图做 DFS。
2. 按完成时间从大到小排列顶点。
3. 得到拓扑序。

Kahn 算法做法:

1. 统计所有顶点入度。
2. 将入度为 0 的顶点入队。
3. 每取出一个顶点, 就删除它的出边并更新邻点入度。
4. 新的入度为 0 的顶点入队。

若最终输出顶点数少于 v , 说明图中存在环。

任务调度

任务依赖关系可建模为有向图:

$u \rightarrow v$ 表示任务 u 必须先于任务 v 完成

若图是 DAG, 则拓扑序给出合法任务执行顺序。

若图有环, 则依赖关系冲突, 无法完成全部任务。

关键路径

关键路径通常在有向无环图中讨论。边或点带有持续时间。

基本步骤：

1. 求拓扑序。
2. 按拓扑序计算每个点的最早开始或最早完成时间。
3. 反向按逆拓扑序计算最晚开始或最晚完成时间。
4. 松弛时间为 0 的活动位于关键路径上。

关键路径长度决定项目最短完成时间。

强连通区域问题

有向图中，若两个顶点互相可达，则它们属于同一个强连通分量。

Kosaraju 算法：

1. 对原图做 DFS，记录完成时间。
2. 将所有边反向。
3. 按原图完成时间从大到小，在反图上 DFS。
4. 每次 DFS 访问到的一组顶点构成一个强连通分量。

Tarjan 算法也使用 DFS，通过 dfn 和 low 维护强连通分量。

例题

例题 1

题目：给定有向图：

$$V = A, B, C, D, E, F$$

$$E = A \rightarrow C, B \rightarrow C, B \rightarrow D, C \rightarrow E, D \rightarrow F, E \rightarrow F$$

写出一个拓扑序。

解答：

一个合法拓扑序为：

A, B, C, D, E, F

检查每条边：

A 在 C 前
B 在 C 前
B 在 D 前
C 在 E 前
D 在 F 前
E 在 F 前

所有依赖均满足，因此该序列是拓扑序。

例题 2

题目：在无权图中从 s 出发做 BFS。边集合为：

$s-a, s-b, a-c, b-c, c-d, b-e$

求从 s 到各点的最短距离。

解答：

BFS 按层推进：

第 0 层： s
第 1 层： a, b
第 2 层： c, e
第 3 层： d

距离为：

$\text{dist}[s] = 0$
 $\text{dist}[a] = 1$
 $\text{dist}[b] = 1$
 $\text{dist}[c] = 2$
 $\text{dist}[e] = 2$
 $\text{dist}[d] = 3$

例题 3

题目：求下面有向图的强连通分量：

$1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1, 3 \rightarrow 4, 4 \rightarrow 5, 5 \rightarrow 4$

解答：

顶点 1, 2, 3 互相可达，构成一个强连通分量：

$\{1, 2, 3\}$

顶点 4, 5 互相可达，构成一个强连通分量：

$\{4, 5\}$

边 $3 \rightarrow 4$ 只提供从第一个分量到第二个分量的方向，不能让两个分量合并。

常见误区

BFS 适合无权图最短路，不适合直接处理带权最短路。

拓扑序只对 DAG 存在。若图有有向环，则不存在拓扑序。

强连通分量是有向图概念，不是无向连通分量。